Implementation Brief — Online Food Delivery System

Project: SCSE2233 Final Project · Group Hub (Group 11) · **Read CLAUDE.md first.**
Purpose: turn the Master SDD into a working, deployed system, class-for-class.

---

0. Prime directive

Build **exactly what the SDD specifies**. The SDD (..\Assignment 3\our answer\Assignment3_Master_SDD_OnlineFoodDelivery.pdf) defines the modules, the 10 data entities, the 9 design patterns, the decision-table logic, and the wireframes. Your code must mirror them so the grader can trace **UI → Decision Table → Pattern → Code** (the "red thread", SDD §6).

---

1. Stack (decided)

- **Backend:** Java 17, Spring Boot 3, Spring Web, Spring Data JPA, Spring Security (JWT), Spring WebSocket (STOMP), PostgreSQL, Maven.
- **Frontend:** React 18 + Vite + TypeScript + Tailwind CSS + Axios + React Router.
- **Payment:** Stripe **test mode** behind the Adapter (StripeGatewayAdapter); if no Stripe key, use a MockGatewayAdapter that returns APPROVED so the demo still runs.
- **Realtime:** STOMP over WebSocket for live delivery tracking.
- **Deploy:** backend → Railway or Render (Docker) · Postgres → same provider or Supabase · frontend → Vercel.
- *(If the team truly prefers Node: Express + TypeScript + Prisma is an acceptable substitute, but keep the same class/pattern names. Java is preferred because the SDD code is Java.)*

2. Repository layout

```
online-food-delivery/            <- the git repo (created/cloned inside "From GitHub")
├─ README.md
├─ .gitignore
├─ docker-compose.yml            <- local Postgres + backend
├─ backend/
│  ├─ pom.xml
│  └─ src/main/java/com/grouphub/ofd/
│     ├─ config/        # ConfigurationManager (SINGLETON), SecurityConfig, WebSocketConfig
│     ├─ auth/          # M1: AuthController, AuthService, AuthenticationStrategy (+Password/OAuth), User, UserRepository, JwtUtil
│     ├─ restaurant/    # M1: RestaurantController, IRestaurantService, RestaurantService, RestaurantCacheProxy (PROXY), Restaurant, MenuItem
│     ├─ cart/          # M2: CartController, CartService, ShoppingCart, CartItem,
```

CartRepository, InventoryService
│   ├─ order/   # M2: OrderController, OrderService, OrderBuilder (BUILDER), CheckoutFacade (FACADE), Order, OrderState (STATE)+states
│   ├─ payment/   # M3: PaymentController, PaymentService, PaymentGatewayFactory (FACTORY)+Stripe/PayPal, PaymentGateway, StripeGatewayAdapter (ADAPTER), Transaction
│   ├─ delivery/   # M3: DeliveryController, DeliveryService, TrackingService (OBSERVER subject), DeliveryObserver+Customer/Notification, Driver
│   └─ common/   # DTOs, enums (OrderStatus, Role…), exceptions (EmptyCartException, InsufficientStockException…)
│ └─ src/main/resources/ # application.yml, schema.sql, data.sql (seed test accounts)
└─ frontend/
  └─ src/
    ├─ pages/   # Login, Browse, Menu, Cart, Checkout, Payment, Tracking, (Admin)
    ├─ components/ api/ store/ App.tsx main.tsx

Keep CLAUDE.md, IMPLEMENTATION_BRIEF.*, the SDD and slides **outside** this repo folder.

3. Database schema (from the SDD ERDs, §5.4) — schema.sql

Ten tables. Types are PostgreSQL.

```
CREATE TABLE app_user (          -- USER
  user_id BIGSERIAL PRIMARY KEY, name VARCHAR(120), email VARCHAR(160) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL, phone VARCHAR(20),
  role VARCHAR(10) NOT NULL CHECK (role IN ('CUSTOMER','ADMIN','DRIVER')),
  created_at TIMESTAMP DEFAULT now());
CREATE TABLE session (           -- SESSION
  session_id BIGSERIAL PRIMARY KEY, user_id BIGINT REFERENCES app_user(user_id),
  jwt_token VARCHAR(512), issued_at TIMESTAMP DEFAULT now(), expires_at TIMESTAMP);
CREATE TABLE restaurant (        -- RESTAURANT
  restaurant_id BIGSERIAL PRIMARY KEY, name VARCHAR(150), cuisine VARCHAR(60),
  rating DECIMAL(2,1), location VARCHAR(255), is_open BOOLEAN DEFAULT true);
CREATE TABLE menu_item (         -- MENU_ITEM
  menu_item_id BIGSERIAL PRIMARY KEY, restaurant_id BIGINT REFERENCES restaurant(restaurant_id),
  name VARCHAR(150), description TEXT, price DECIMAL(10,2), is_available BOOLEAN DEFAULT true);
CREATE TABLE cart (              -- CART
  cart_id BIGSERIAL PRIMARY KEY, user_id BIGINT REFERENCES app_user(user_id),
```

```
  status VARCHAR(12) DEFAULT 'ACTIVE', subtotal DECIMAL(10,2) DEFAULT 0,
updated_at TIMESTAMP DEFAULT now());
CREATE TABLE cart_item (          -- CART_ITEM
  cart_item_id BIGSERIAL PRIMARY KEY, cart_id BIGINT REFERENCES
cart(cart_id),
  menu_item_id BIGINT REFERENCES menu_item(menu_item_id), quantity INT,
unit_price DECIMAL(10,2));
CREATE TABLE orders (             -- ORDERS
  order_id BIGSERIAL PRIMARY KEY, user_id BIGINT REFERENCES
app_user(user_id),
  cart_id BIGINT REFERENCES cart(cart_id), delivery_address VARCHAR(255),
total_amount DECIMAL(10,2),
  status VARCHAR(20) DEFAULT 'PENDING_PAYMENT', created_at TIMESTAMP
DEFAULT now());
CREATE TABLE transaction (        -- TRANSACTION
  transaction_id BIGSERIAL PRIMARY KEY, order_id BIGINT REFERENCES
orders(order_id),
  gateway_ref VARCHAR(120), amount DECIMAL(10,2), currency CHAR(3)
DEFAULT 'MYR',
  status VARCHAR(10), paid_at TIMESTAMP);
CREATE TABLE driver (             -- DRIVER
  driver_id BIGSERIAL PRIMARY KEY, name VARCHAR(120), phone
VARCHAR(20),
  vehicle VARCHAR(60), is_active BOOLEAN DEFAULT true);
CREATE TABLE delivery (           -- DELIVERY
  delivery_id BIGSERIAL PRIMARY KEY, order_id BIGINT REFERENCES
orders(order_id),
  driver_id BIGINT REFERENCES driver(driver_id), status VARCHAR(20)
DEFAULT 'PREPARING',
  current_lat DECIMAL(9,6), current_lng DECIMAL(9,6), eta TIMESTAMP);
```

4. The 9 design patterns — implement under these EXACT names (SDD §5.6)

Module	Creational	Structural	Behavioural
M1	ConfigurationManager (Singleton)	RestaurantCacheProxy (Proxy)	AuthenticationStrategy + PasswordAuthStrategy, OAuthStrategy (Strategy)
M2	OrderBuilder (Builder)	CheckoutFacade (Facade)	OrderState + PendingPaymentState,ConfirmedState,OutForDeliveryState,DeliveredState,CancelledState (State)
M3	PaymentGatewayFactory +	StripeGatewayAdapter over	TrackingService subject +

StripeGatewayFactory,PayPalGatewayFactory (Factory Method)	StripeExternalApi (Adapter)	DeliveryObserver →CustomerTrackingClient,NotificationObserver (Observer)

The SDD §5.6.2–5.6.4 already contains the Java for these — port it, expand it into full Spring components, keep the comments that map to the decision tables.

5. Module build tasks (each traces to the SDD)

M1 — User Auth & Restaurant Browsing (Mohammed Alsakkaf)

- **Entities/repos:** User, Session, Restaurant, MenuItem (+ JPA repositories).
- **Auth:** register + login → BCrypt hash, issue JWT (HttpOnly). Implement **DT-M1-1** (login) and **DT-M1-2** (registration) exactly, including the lock-after-5-attempts and duplicate-email paths.
- **Browsing:** RestaurantCacheProxy in front of RestaurantService with a 60s TTL cache → **DT-M1-3** (cache hit/miss, no results, closed restaurant).
- **Screens (wireframes §5.5.2):** Login/Register, Browse (search + filter chips + result cards), Menu.
- **Endpoints:** POST /api/auth/register, POST /api/auth/login, GET /api/restaurants?loc&cuisine&rating, GET /api/restaurants/{id}/menu.

M2 — Shopping Cart & Order Placement (Anas Ahmed Hisham)

- **Entities/repos:** Cart, CartItem, Order.
- **Cart:** add/update items with InventoryService.checkAvailability → **DT-M2-1** (qty 1..20, unavailable, duplicate→increment).
- **Checkout:** CheckoutFacade.checkout() orchestrates validateCart → reserveInventory → OrderBuilder.build() → Order starts in PendingPaymentState → **DT-M2-2** (empty cart, missing address, out-of-stock, success).
- **Screens (§5.5.3):** Cart, Checkout.
- **Endpoints:** POST /api/cart/items, PATCH /api/cart/items/{id}, GET /api/cart, POST /api/checkout.

M3 — Payment Processing & Delivery Tracking (Imtiaj Ahmed)

- **Entities/repos:** Transaction, Delivery, Driver.
- **Payment:** PaymentGatewayFactory.createGateway() → StripeGatewayAdapter.chargeCard() → record Transaction → Order transitions to ConfirmedState → **DT-M3-1** (not payable, invalid amount, timeout→PENDING, approved, declined). No card data stored (PCI).
- **Tracking:** TrackingService (Observer subject) pushes GPS every 10s over WebSocket to CustomerTrackingClient → **DT-M3-2** (awaiting driver, delivered, GPS lost→last known).
- **Screens (§5.5.4):** Payment (tokenised card), Live Tracking (map + ETA).

- **Endpoints:** POST /api/payments, GET /api/orders/{id}, WS /ws/tracking (subscribe /topic/order/{id}).
- **State machine (§5.8.5):** PENDING_PAYMENT → CONFIRMED → OUT_FOR_DELIVERY → DELIVERED; PENDING_PAYMENT → CANCELLED.

6. Seed data — data.sql (the test accounts for the Dr's slide)

Insert with **BCrypt-hashed** passwords: - Customer — customer@foodhub.test / Customer@123 - Admin — admin@foodhub.test / Admin@123 - Delivery Driver — driver@foodhub.test / Driver@123 Plus ~5 restaurants, ~15 menu items, and 2 drivers so the demo has content.

7. Build order (do NOT skip steps; verify each)

1. Scaffold repo (backend Spring Boot + frontend Vite), .gitignore, docker-compose.yml, README.md.
2. Wire Postgres + JPA + schema.sql + data.sql; confirm the app boots and tables exist.
3. **M1** backend → test endpoints (register/login/browse) with curl/Postman → then M1 frontend screens.
4. **M2** backend → test cart/checkout → then M2 screens.
5. **M3** backend → test payment (mock gateway) + WebSocket tracking → then M3 screens.
6. Integration pass: full customer journey login → browse → cart → checkout → pay → track.
7. Deploy backend + DB, then frontend; wire the frontend to the live API URL.
8. Verify the 3 seeded logins work on the **live** URL.
9. Fill the real GitHub + live URLs + confirmed credentials into **SDD Appendix C** and **the slide**.

8. Teammate contribution (per the owner's plan)

The code is organised by module (M1/M2/M3). The owner (Mohammed) will have each teammate push **their own module's files** under **their own GitHub account**, following the same commit laws (one file, ≤50–100 lines, their identity). Optionally stage each teammate's module files in From GitHub\for-teammates\M2-Anas\ and M3-Imtiaj\ (outside the repo) for easy handoff. **Do not fabricate teammate commits yourself.**

9. README must include

Local run (docker-compose + mvn spring-boot:run + npm run dev), env vars (DATABASE_URL, JWT_SECRET, STRIPE_KEY optional), deploy steps, and the seeded test accounts.

10. Acceptance checklist (Definition of Done)

- ☐ All 10 tables created and seeded · 3 role logins work.
- ☐ All 9 patterns present under the SDD names.

- ☐ Every decision table (DT-M1-1 … DT-M3-2) enforced, including edge cases.
- ☐ Full journey works locally and on the live URL.
- ☐ Commit history: owner identity only, one file per commit, ≤100 lines each, grouped by module.
- ☐ README complete · Appendix C + slide updated with real URLs.